

Heap Sort: An Efficient Sorting Algorithm

Bhattacharya Brothers (Darpan Bhattacharya (B2430044) & Soham Bhattacharya (B2430059))

August 28, 2024

Introduction

Heap Sort is a comparison-based sorting algorithm that builds on the concept of a binary heap data structure. It offers an efficient and reliable way to sort a list of elements, particularly suitable for large datasets. In this article, we will explore how Heap Sort works, its properties, and its advantages.

The Problem

Given an array $A[0..n-1]$ we need to sort it in non-decreasing order. An array $A[0..n-1]$ is said to be sorted in non-decreasing order if the condition $A[0] \leq A[1] \leq \dots \leq A[n-1]$ holds.

Understanding the Binary Heap

To grasp Heap Sort, it is essential to understand the binary heap. A binary heap is a complete binary tree that satisfies the heap property. There are two types of binary heaps:

1. **Max-Heap:** In a max-heap, the value of each parent node is greater than or equal to the values of its children. The largest element is found at the root.
2. **Min-Heap:** In a min-heap, the value of each parent node is less than or equal to the values of its children. The smallest element is found at the root.

Heap Sort typically uses a max-heap to sort an array in ascending order. The basic idea is to build a max-heap from the input data and then repeatedly extract the maximum element from the heap, rebuilding the heap until it is empty.

Heap Sort Algorithm

The Heap Sort algorithm consists of two main phases:

1. **Building the Max-Heap:**
 - Convert the array into a max-heap. This step ensures that the largest element is at the root of the heap.
 - Starting from the middle of the array (since leaves are already heaps), heapify each element by comparing it with its children and swapping if necessary. This process is called heapification.
2. **Sorting:**
 - Swap the root (the largest element) with the last element of the heap. This action removes the maximum element from the heap.

- Reduce the size of the heap by one and repeat the heapification process for the new root. This step ensures that the heap property is maintained.
- Continue this process until all elements are extracted and the array is sorted.

Pseudocode for Heap Sort

```

HeapSort(arr):
    BuildMaxHeap(arr)
    for i from len(arr) - 1 to 1:
        swap(arr[0], arr[i])
        heapify(arr, 0, i)

BuildMaxHeap(arr):
    for i from len(arr) // 2 - 1 to 0:
        heapify(arr, i, len(arr))

heapify(arr, i, heap_size):
    largest = i
    left = 2 * i + 1
    right = 2 * i + 2

    if left < heap_size and arr[left] > arr[largest]:
        largest = left

    if right < heap_size and arr[right] > arr[largest]:
        largest = right

    if largest != i:
        swap(arr[i], arr[largest])
        heapify(arr, largest, heap_size)

```

Time and Space Complexity

Heap Sort operates with a time complexity of $O(n \log n)$ for all cases (best, average, and worst). This efficiency arises from the logarithmic time required for the heapification process and the linear number of elements that need to be heapified. The space complexity of Heap Sort is $O(1)$ because it is an in-place sorting algorithm, requiring only a constant amount of extra space.

Advantages and Disadvantages

Advantages:

- Heap Sort is not sensitive to the initial order of the elements, making it a robust sorting algorithm.
- It is an in-place algorithm, meaning it does not require additional storage space.
- The algorithm guarantees $O(n \log n)$ time complexity in all cases.

Disadvantages:

- Heap Sort is not a stable sort, meaning the relative order of equal elements may not be preserved.
- While efficient, it is often slower in practice compared to other algorithms like Quick Sort due to larger constant factors in its time complexity.

Conclusion

Heap Sort is a powerful and efficient sorting algorithm, especially suitable for large datasets where $O(n \log n)$ performance is required. Its use of a binary heap structure ensures that it remains reliable across different cases, making it a valuable tool in the programmer's toolkit.